Lab 08 Command Pattern, Undo, Redo

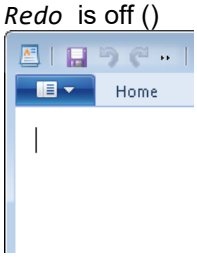
Q0:Introduction I - How users use Undo and Redo
II - An arraylist as a static field

Q1 - Q6 Maintenance of `Employee` records with undo-redo commands
Q1 : The start-up version - `addSalary` + `listAllRecords`
Q2 : + `undo` + `redo`
Q3-6 [homework] : `changeAnnualLeaves`(Q3), `exchangeSalaries`(Q4), `fire` Lazyguy(Q5)
Checking of undo/redo lists (Q6)

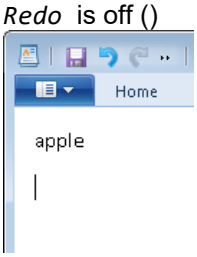
Q0.I How users use Undo and Redo

Read the following undo/redo example.

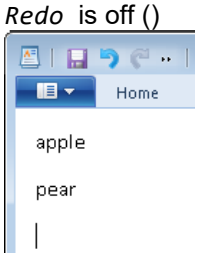
Step 1. Start Wordpad
Undo is off ()
Redo is off ()



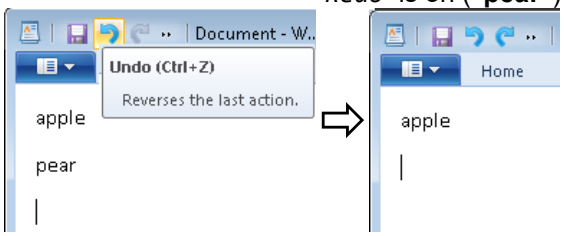
Step 2. Type "apple<enter>"
Undo is on (**"apple"**)
Redo is off ()



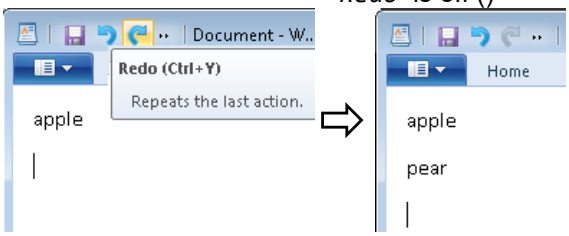
Step 3. Type "pear<enter>"
Undo is on ("apple", **"pear"**)
Redo is off ()



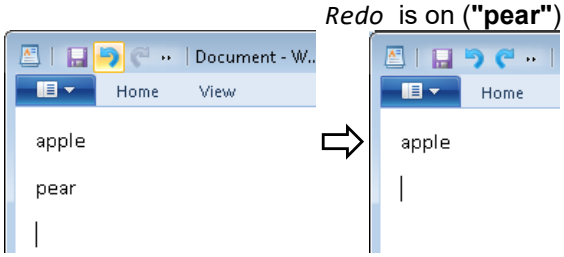
Step 4. Click *undo*
Undo is on ("apple")
Redo is on (**"pear"**)



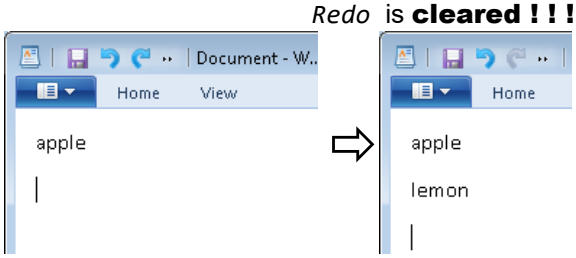
Step 5. Click *redo*
Undo is on ("apple", **"pear"**)
Redo is off ()



Step 6. Click *undo*
Undo is on ("apple")
Redo is on (**"pear"**)



Step 7. Type "lemon<enter>"
Undo is on ("apple", **"lemon"**)
Redo is **cleared ! ! !**



Your task: Circle the correct choice in each question.

a) undo is available only when the undo-list / redo-list is holding 1 or more items.
b) redo is available only when the undo-list / redo-list is holding 1 or more items.
c) Each time when we undo, the undone action is added to the undo-list / redo-list. (steps 4,6)
d) Each time when we redo, the redone action is added back to the undo-list / redo-list. (steps 5)
e) Each time when we edit the content, the action is added to the undo-list / redo-list.
f) [continue (e)] and, the undo-list/redo-list is cleared (see step 7).

Q0.II An arraylist as a static field

Study the given program.

(i) Write down the output,

(ii) Draw all fields and objects which appear during program execution.

```
class Pocket
{
  //Instance field (Note: Object fields are first initialized to null by java)
  private Object thing;

  //Static arraylist
  private static ArrayList<Pocket> createdPockets = new ArrayList<>();

  //Constructor
  public Pocket(Object t)
  {
    this.thing=t;
    createdPockets.add(this);
  }

  @Override
  public String toString() {return thing.toString();}

  public static void listEverything()
  {
    for (Object p:createdPockets)
      System.out.println(p);
  }
}
```

```
public class Main_Lab08Q00
{
  public static void main(String [] args)
  {

    new Pocket("Pencil");
    new Pocket(2014);
    new Pocket("Spring");

    Pocket.listEverything();

  }
}
```

Output:

Drawing:

Q1 Employee Program - The start-up version: `addSalary` + `listAllRecords`

Create a program which changes the salaries of employees in a company.

Sample input file:

First line is number employees →

Then the data of employees:
- name,
- salary,
- no. of days of annual leaves entitled

Remaining lines contain user commands:
- Add salary: **`addSalary`** *`name`* `by_amount`
- Listing: **`list`**

Blank lines added for easy reading.

e1.txt

```
3
Helena 10000 7
Jason 20000 21
Kit 30000 21

list

addSalary Helena 5000
addSalary Jason 2000

list
```

Sample rundown:

Rundown 1

```
Please input the file pathname: c:\e1.txt

> list
Helena ($10000, 7 days)
Jason ($20000, 21 days)
Kit ($30000, 21 days)

> addSalary Helena 5000
Done.

> addSalary Jason 2000
Done.

> list
Helena ($15000, 7 days)
Jason ($22000, 21 days)
Kit ($30000, 21 days)
```

Implementation (Also see **given code**):

Step 1. Finish the **Employee class**:

Instance fields:
```
private String name;
private int salary;
private int annualLeaves;
```

Methods:
```
public Employee(/* 3 parameters*/) {..} //constructor
public String getName(){..}
public void addSalary(int increase) {..}
public String toString() {..} //Return a string like: " Kit ($30000, 21 days)" [given_code.txt]
```

add @Override

Step 2. Finish the **Company class** [Implemented as singleton *]

Instance fields: `private ArrayList<Employee> allEmployees;` //Arraylist of all employees

↖import java.util.*

Methods:
```
private Company() {..} //* private constructor: allEmployees = new ArrayList<>();
public void listEmployees() {..} //for each employee, apply.toString() and output.
public Employee findEmployee(String name) {..} //search an employee
public void addEmployee(Employee e) {..} //add an employee to the array list
```

Static field for Singleton * : `private static Company instance = new Company();`
Static method for Singleton * : `public static Company getInstance() {return instance;}`

Step 3. Finish the **main** method in the **Main class**

(i) Add employee records into the company

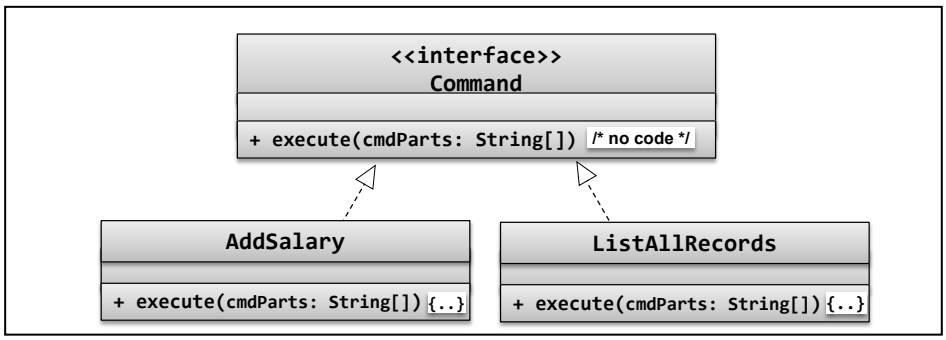
(ii) Handle each command using `AddSalary` and `ListAllRecords` (will be implemented next)

[Study and apply `given_code.txt`]

Step 4. Create `Command.java, AddSalary.java, ListAllRecords.java`:

[`Command.java`]: The `Command` interface which contains only one method called `execute`.

[`AddSalary.java`] and [`ListAllRecords.java`]: The classes which process user commands by implementing the `execute` method.



4.1 Finish the `Command` interface:

`Command.java`

```
interface Command
{
                          ← void execute(String[] cmdParts);
}
```

4.2 Finish the `AddSalary` class:

`AddSalary.java`

```
public class AddSalary implements Command
{
                          ←@Override
                           public void execute(String[] cmdParts)
                           {
                               Employee e;
                               int addAmount;

                               Company company = Company.getInstance();
                               e = company.findEmployee(cmdParts[1]);
                               addAmount=Integer.parseInt(cmdParts[2]);

                               e.addSalary(addAmount);
                               System.out.println("Done.");
                           }
}
```

Please study Steps 4.1 amd 4.2 carefully.

4.3 Finish the `ListAllRecords` class on your own. (Note: `execute`() is short: 2 lines only)

```
                    ↑
Use the listEmployees() method of Company.
```

Q2 `addSalary` + `listAllRecords` **+ undo + redo**

Read the sample input and rundown on the right.

We change the command handling module:

- Add the `RecordedCommand` abstract class as a sub-type of `Command`.
  - `RecordedCommand` will be the superclass for undo/redoable commands:
    `AddSalary(Q2)`,
    `ChangeAnnualLeaves` (Q3),
    `ExchangeSalaries` (Q4),
    `Fire` Lazyguy (Q5)
  - `RecordedCommand` uses 2 static arraylists (Like `createdPockets` see Page 2) as *`undolist`*, *`redolist`*.
- `ListAllRecord` does not undo/redo. So it remains implementing the `Command` interface.

e2.txt

```
3
Helena 10000 7
Jason 20000 21
Kit 30000 21

list

addSalary Helena 5000
addSalary Jason 2000

list

undo
list

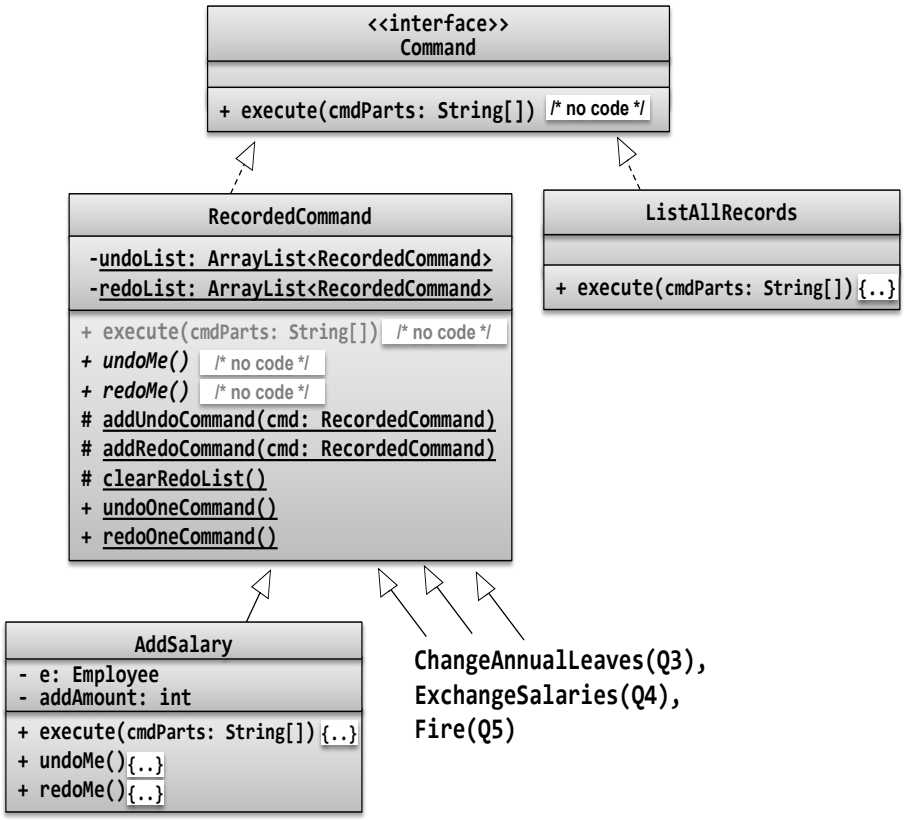
undo
list

redo
list

redo
list
```

Rundown

```
Please input the file pathname: c:\e2.txt
> list
Helena ($10000, 7 days)
Jason ($20000, 21 days)
Kit ($30000, 21 days)
> addSalary Helena 5000
Done.
> addSalary Jason 2000
Done.
> list
Helena ($15000, 7 days)
Jason ($22000, 21 days)
Kit ($30000, 21 days)
> undo
> list
Helena ($15000, 7 days)
Jason ($20000, 21 days)
Kit ($30000, 21 days)
> undo
> list
Helena ($10000, 7 days)
Jason ($20000, 21 days)
Kit ($30000, 21 days)
> redo
> list
Helena ($15000, 7 days)
Jason ($20000, 21 days)
Kit ($30000, 21 days)
> redo
> list
Helena ($15000, 7 days)
Jason ($22000, 21 days)
Kit ($30000, 21 days)
```



Implementation

Please refer to the following files available at the course web:
given_code (`main()` and `addSalary()`), input files, and expected rundown

Please refer to the given code of `RecordedCommand` on next page.

RecordedCommand.java

```
import java.util.*;

public abstract class RecordedCommand implements Command
{
    /* 2 abstract methods for subclasses to implement */ ⑤
    public abstract void undoMe();
    public abstract void redoMe();


    /* undolist and redolist; */ ①
    private static ArrayList<RecordedCommand> undoList=new ArrayList<>();
    private static ArrayList<RecordedCommand> redoList=new ArrayList<>();

    /* adding command to the list */ ②
    protected static void addUndoCommand(RecordedCommand cmd) {undoList.add(cmd);}
    protected static void addRedoCommand(RecordedCommand cmd) {redoList.add(cmd);}

    /* clear redo list*/ ③
    protected static void clearRedoList() {redoList.clear();}

    /* carry out undo/redo */
    public static void undoOneCommand() {undoList.remove(undoList.size()-1).undoMe();} ④
    public static void redoOneCommand() {redoList.remove(redoList.size()-1).redoMe();}
}
```

[Lab08 – Progress] Complete the following blanks:

- We do not *undo* or *redo* any *ListAllRecords* command, so it _____(is / is not) a kind of `RecordedCommand`.

① Each *AddSalary* command is handled as _____ (an object / a class), recorded in _____ (arraylists / arrays).
In Q3-Q5, *ChangeAnnualLeaves, ExchangeSalaries*, *FireLazyguy* will be recorded _____(similarly / differently.

There are _____(only 2 / a lot of) arraylists for recording these commands. These arraylists :

- are maintained by the _____(Command / RecordedCommand) class.
- are managed using _____(fields of object instances / fields of the class) of the `RecordedCommand` class
- are created when the `RecordedCommand` class is loaded.

② to ④ are the methods to work with the command objects stored in ①:

- Grab a command in the undo list and undo it; Grab a command in the redo list and redo it: _____ (②/③/④)
- Add a command to the undo list; Add a command to the redo list: _____ (②/③/④)
- Clear the redo list: _____ (②/③/④)

⑤ `undoMe()` and `redoMe()` are _____(abstract/concrete) methods of `RecordedCommand`.
For different command objects (AddSalary, ChangeAnnualLeaves etc..), undoMe or redoMe should do _____(different/same) actions. Thus specific code are required in each concrete subclass: `AddSalary`, `ChangeAnnualLeaves` etc..

For Q3-Q6, refer to the given input files (e3.txt, e4.txt, e5.txt, e6.txt) and rundown samples (rundown3, ..)

Q3 [Homework] + `changeAnnualLeaves`

Q4 [Homework] + `exchangeSalaries`

Q5 [Homework] + `fire` Lazyguy

← Implement these commands in ChangeAnnualLeaves.java, ExchangeSalaries.java, Fire.java.
Add handling in Main.java
You may add methods in the Employee and Company classes.

Q6 [Homework] + checking of undo-redo list

Show `"Nothing to undo."`, `"Nothing to redo."` when the undo/redo commands see empty undo/redo lists.

Self-check:
You should only need to modify `undoOneCommand()` and `redoOneCommand()` inside `RecordedCommand.java`.

-- END --